



# Binary Search Optimization

THE DIVIDE-AND-CONQUER RULE

**Module 09:** Advanced Search Logic

**Series:** CodeHive Computational Series

# 01. The Logarithmic Power

---

**Binary Search** is one of the most efficient algorithms ever conceived for data retrieval. Unlike Linear Search, which visits every item, Binary Search works by repeatedly halving the search area.

**The Mandatory Prerequisite:** Binary Search only works on data that is already **sorted**. If the data is random, the "Go Left or Go Right" logic fails immediately.

## Why it is Fast

Imagine searching for a word in a 1,000-page dictionary. You don't start at page 1 (Linear Search); you open it to the middle. If your word starts with "S" and the middle page is "M", you ignore the first 500 pages completely. You just halved your workload in one second.

## 02. Operational Mechanism

---

The algorithm uses three pointers: **Left** (start of range), **Right** (end of range), and **Mid** (the current probe).

### The Binary Protocol:

- 01 Calculate the middle index:  $(\text{Left} + \text{Right}) // 2$ .
- 02 Check if Mid is the target. If yes, stop.
- 03 If Target is smaller than Mid, discard the right half by setting  $\text{Right} = \text{Mid} - 1$ .
- 04 If Target is larger than Mid, discard the left half by setting  $\text{Left} = \text{Mid} + 1$ .
- 05 Repeat until the pointers cross or the item is found.

## 03. Manual Run-Through

Target Value: **11** in array [2, 3, 7, 7, 11, 15, 25]

### Search Phases:

- **Phase 1:** Range 0-6. Mid is index 3 (value 7).  
 $7 < 11$ . Search Right.
- **Phase 2:** Range 4-6. Mid is index 5 (value 15).  
 $15 > 11$ . Search Left.
- **Phase 3:** Range 4-4. Mid is index 4 (value 11).  
**Match Found!**

**Efficiency Note:** In just 3 comparisons, we found our target in a list of 7 items. In a list of 1,000,000 items, we would find it in only 20 comparisons.

## 04. Python Implementation

Implementing Binary Search requires an iterative loop that narrows the boundaries on every cycle.

```
def binarySearch(arr, targetVal):
    left = 0
    right = len(arr) - 1

    while left ≤ right:
        mid = (left + right) // 2

        # Found the target
        if arr[mid] == targetVal:
            return mid

        # Narrow the search area
        if arr[mid] < targetVal:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

## 05. Time Complexity Analysis

The beauty of Binary Search lies in its logarithmic growth. As the input size doubles, the time taken only increases by a single step.

$O(\log_2 n)$

### Comparison vs. Linear Search

| Elements (n)  | Linear Search (n) | Binary Search (log n) |
|---------------|-------------------|-----------------------|
| 1024          | 1,024 steps       | 10 steps              |
| 1,048,576     | ~1.05M steps      | 20 steps              |
| 1,073,741,824 | ~1.07B steps      | 30 steps              |

## 06. Developer's Checklist

---

To use Binary Search effectively in production environments, keep these constraints in mind:

- **Sorting Overhead:** Sorting takes  $O(n \log n)$ . Do not sort just to search once. Use Binary Search if you perform many searches on the same dataset.
- **Memory Access:** Binary Search requires "Random Access" (indexing). It works perfectly on arrays/lists, but poorly on Linked Lists (due to traversal needs).
- **Integer Overflow:** In some languages,  $(left + right) // 2$  can overflow. In Python, this is not an issue due to arbitrary-precision integers.

---

**CODEHIVE MASTERY**

© 2026. Data Algorithms Research Group.